

LOAN APPROVAL MLOPS



ระบบคัดกรอง คำขอสินเชื่อ ตั้งแต่ข้อมูลดิบ จนถึงการให้บริการพร้อมเผื่อระวัง และเทรนใหม่อัตโนมัติ

สมาชิก

- | | | |
|---------------------------|-------------|-------|
| 1.นายอภิมุข เพี้ยสักขา | 673380354-9 | SEC.1 |
| 2.นายณภัทร โสตาตง | 673380076-1 | SEC.1 |
| 3.นางสาวเตชินี ฤามูลตรี | 673380358-1 | SEC.1 |
| 4.นายจักรภัทร เอี่ยมเสริม | 673380309-4 | SEC.2 |
| 5.นายพุฒิพงศ์ พานิชพันธุ์ | 673380335-3 | SEC.2 |



01 · ปัญหาและที่มา

ปัญหา

ธนาคารได้รับคำขอสินเชื่อจำนวนมาก และทุกคำขอต้องมีเจ้าหน้าที่พิจารณา ทำให้เกิดปัญหา 3 อย่าง

1. **ความล่าช้า:** ลูกค้านรอนาน เจ้าหน้าที่เสียเวลากับเคสที่ชัดเจนอยู่แล้ว (ผ่านแน่ หรือไม่ผ่านแน่)
2. **ไม่สม่ำเสมอ:** คำขอคล้ายกันอาจได้ผลต่างกัน ขึ้นกับคนพิจารณา
3. **ผิดแล้วแพง และแพงไม่เท่ากัน:** อนุมัติคนที่ไม่ควรได้เสี่ยงเป็นหนี้เสียทั้งก้อน ส่วนปฏิเสธคนที่ควรได้แค่เสียโอกาสได้ดอกเบี้ย

เราจึงตั้งต้นทุนไว้ 3 ต่อ 1

แล้วทำไมถึงต้องทำให้เป็นแบบ mlops?

เพราะโมเดลสินเชื่อไม่ใช่งานที่ทำครั้งเดียวจบ ต้องดูแลต่อเนื่องหลังใช้งานจริง

1. **ข้อมูลเปลี่ยนแปลง:** ลูกค้าและเศรษฐกิจเปลี่ยน โมเดลจะแม่นยำลงเรื่อย ๆ (drift) จึงต้องเทรนใหม่อัตโนมัติ
2. **ต้องตอบทันที:** ลูกค้านรอนานหน้าเคาน์เตอร์/แอป ระบบต้องให้บริการผ่าน API ตลอดเวลา และรู้ทันทีเมื่อช้าหรือล่ม
3. **ตรวจสอบได้และเป็นธรรม:** ย้อนดูได้ว่าโมเดลเวอร์ชันไหนเป็นคนตัดสินใจ และไม่ปฏิเสธกลุ่มใดมากเกินไป

สรุป: MLOps ทำให้วงจร เทรน → ใช้งาน → เฝ้าดู → เทรนใหม่ เกิดขึ้นอัตโนมัติ ไม่ต้องพึ่งคนทำมือ

02 · AI Project Canvas

Data · ข้อมูล - ประวัติคำขอสินเชื่อ 30,000 รายการ 29 features: ตัวเลข 17, หมวดหมู่ 10, อัตราส่วนที่สร้างเอง 2 - ตัด leakage และข้อมูลอ่อนไหวออก - ผลอนุมัติใหม่เข้ามาต่อเนื่อง	Skills · ทักษะที่ต้องใช้ - Machine Learning (binary classification) - MLOps: เทรนใหม่อัตโนมัติ และ deploy เป็น API - Monitoring (Prometheus) Output · ตัวชี้วัด - ทำนาย Loan Approved (Approved / Rejected) - วัดผลด้วย AUC และต้นทุนผิดพลาด 3:1	Value Proposition - ลดเวลาพิจารณาจากหลายวันเหลือไม่กี่วินาที (สำหรับคำขอที่ชัดเจน) - เจ้าหน้าที่ที่มีเวลาดูเคสกำลังมากขึ้น - ตัดสินใจ 3 แบบตามความมั่นใจ: อนุมัติอัตโนมัติ / ปฏิเสธอัตโนมัติ / ส่งให้เจ้าหน้าที่	Integration · ใช้งาน - Real-time API: /predict (เคาน์เตอร์ / แอป) - Batch API: คำขอล้าง - Retrain ทุกสัปดาห์ และเมื่อเกิด drift Stakeholders - เจ้าหน้าที่สินเชื่อ - ฝ่ายบริหารความเสี่ยง - ฝ่ายกำกับดูแล (ดูแลความเป็นธรรม)	Customers · ผู้ใช้ - ผู้ยื่นกู้: ได้ผลเร็วขึ้น ไม่ต้องรอหลายวัน - เจ้าหน้าที่สินเชื่อ: ใช้ผลทำนาย ช่วยตัดสินใจ
Cost · ต้นทุน - อนุมัติผิด (เกิดหนี้เสีย) แพงกว่าปฏิเสธผิด 3 เท่า (ต้นทุนความผิดพลาด 3:1) - เทรนใหม่ทุกสัปดาห์: 3 โมเดลขนานกัน ใช้เวลาไม่ถึง 1 นาที - ตรวจ drift: data drift (KS/Chi2) และ concept drift (AUC ตก) - ติดตาม latency/error (Prometheus) และสัดส่วนการปฏิเสธแยกกลุ่ม		Revenue · ผลตอบแทน - ลดต้นทุนความผิดพลาด 23% (1,879 → 1,446) เทียบกับ threshold 0.5 - ประหยัดเวลาเจ้าหน้าที่: คำขอที่ชัดเจนไม่ต้องรอคนพิจารณา - ลูกค้าได้ผลเร็ว ลดโอกาสเสียลูกค้าให้คู่แข่ง		

03 · การวางกรอบปัญหา

ให้ระบบตัดสินใจ คำขอที่ชัดเจน เจ้าหน้าที่ดูแลเฉพาะเคสกำลัง

ใครได้ประโยชน์

ผู้ยื่นกู้ได้ผลเร็วขึ้น · เจ้าหน้าที่ ดูแลเฉพาะ 22% ที่กำลัง

ทำไมต้อง ML แทนกฎ

CREDIT SCORE ตัวเดียว ได้ AUC 0.77 · โมเดลจาก 27 ฟีเจอร์ ได้ AUC 0.89 และเทรนใหม่เองได้ เหมือนนโยบายเปลี่ยน

ถ้าโมเดลทายผิด

อนุมัติผิดคน = เสียหนี้เสีย

ปฏิเสธผิดคน = เสียลูกค้าและรายได้

04 • ตัวชี้วัด

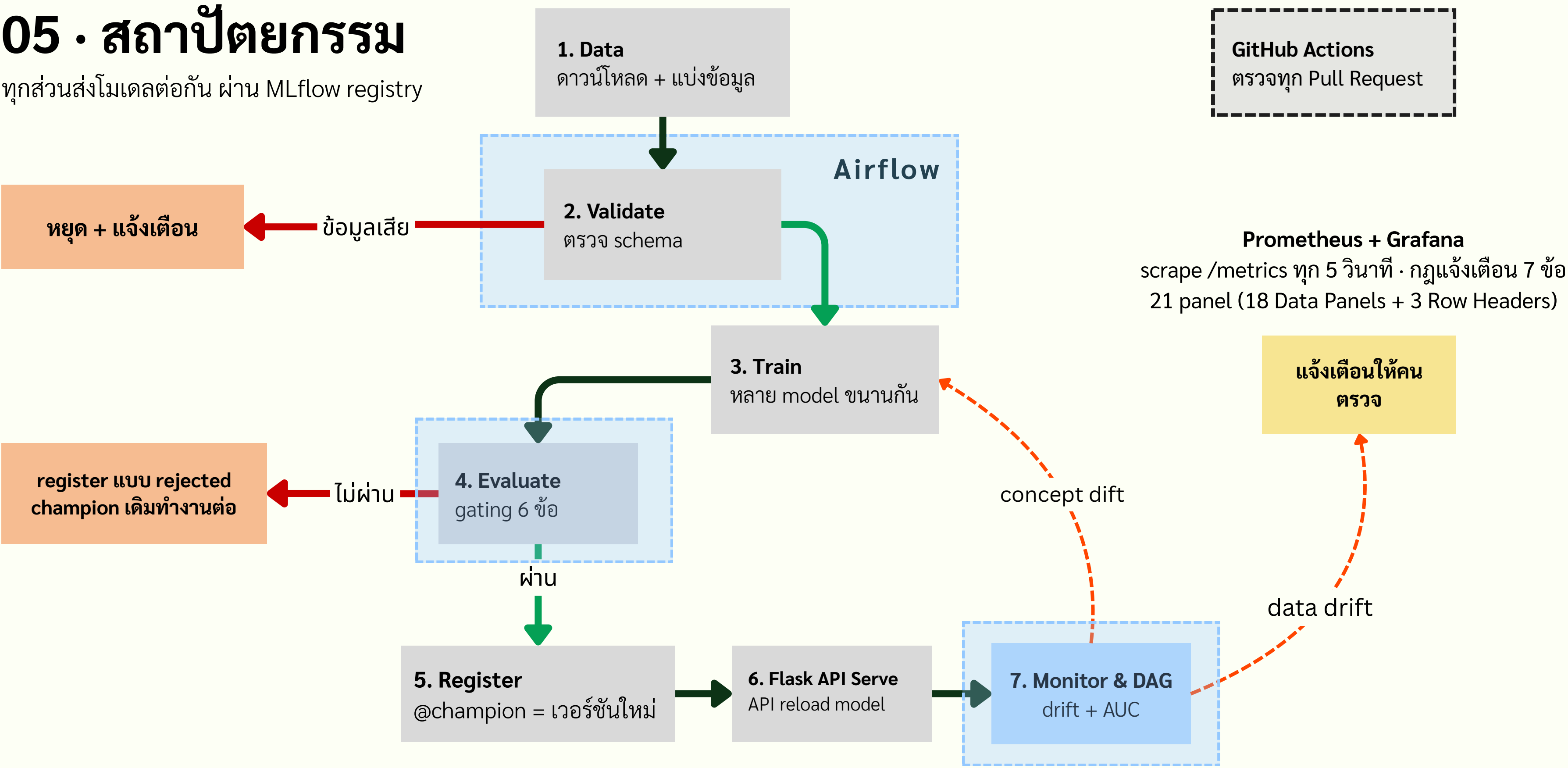
ประเภท	ตัวชี้วัด	เกณฑ์	เหตุผล
Optimizing	ROC-AUC (validation)	ให้มีค่าสูงสุด	วัดการจัดลำดับ ไม่ขึ้นกับ threshold
Gating	ROC-AUC (test)	≥ 0.85	ต่ำกว่านี้ ตัดสินอัตโนมัติได้น้อย
Gating	Recall คลาสปฏิเสธ	≥ 0.80	กันอนุมัติผิดคน
Gating	เทียบ champion เดิม	แย่ลง ≤ 0.003	กันสลับไปโมเดลที่แย่ลง
Gating	p95 latency ต่อ 1 คำขอ	$\leq 25\text{ ms}$	มาจาก SLO 200 ms ที่ 20 req/s
Gating	ขนาดไฟล์โมเดล	$\leq 50\text{ MB}$	reload และ rollback เร็ว
Gating	ส่วนต่างอนุมัติ ชาย-หญิง	≤ 0.10	ความเป็นธรรม

ตัวชี้วัดธุรกิจ: % คำขอที่ตัดสินอัตโนมัติ · ต้นทุนผิดพลาด (อนุมัติผิด 3 : ปฏิเสธผิด 1) ใช้เลือก threshold · เวลาตอบลูกค้า

AUC ยิ่งสูงยิ่งดี ส่วนอีก 6 ด้าน ต้องผ่านทุกข้อ

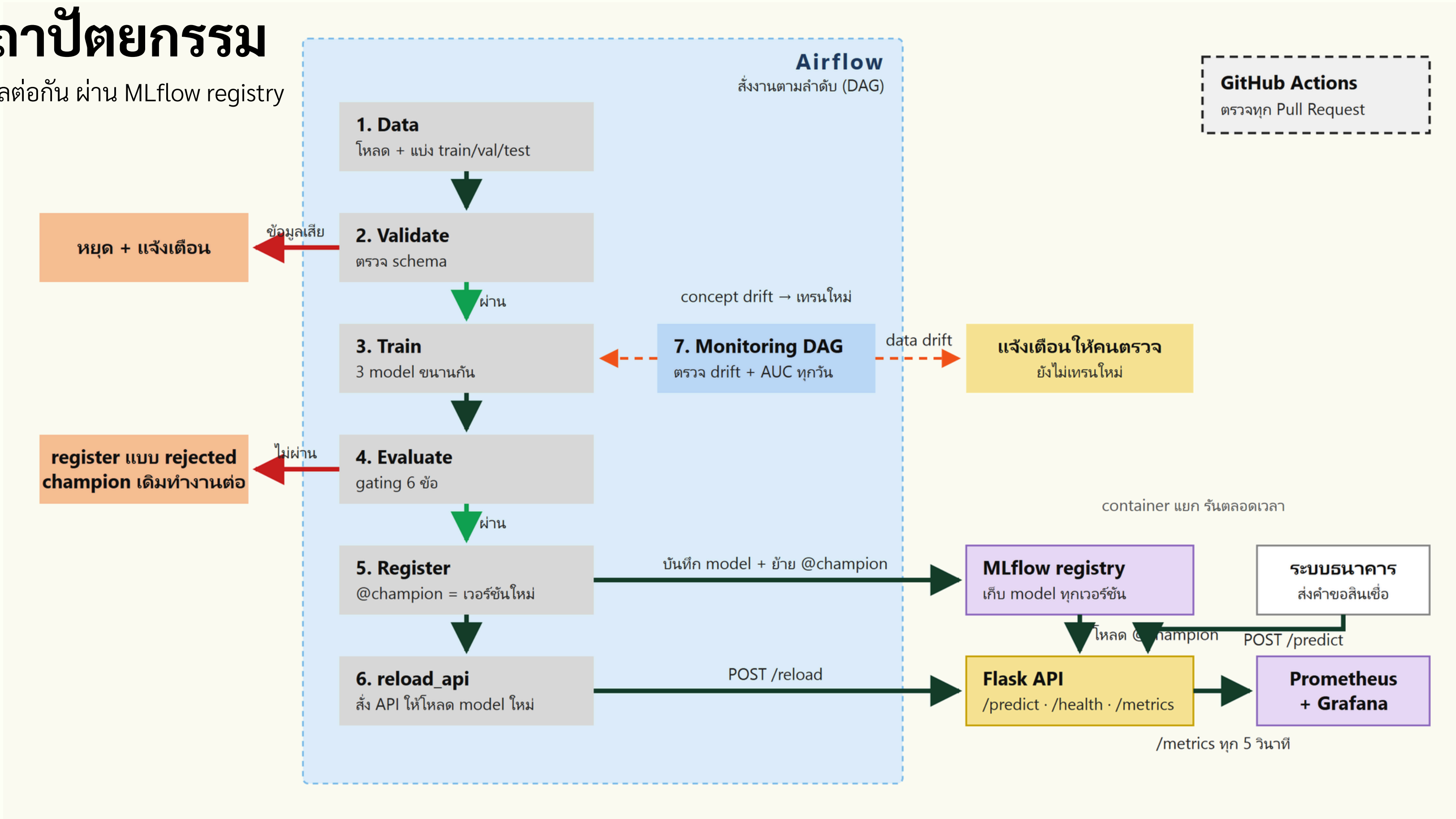
05 · สถาปัตยกรรม

ทุกส่วนส่งโมเดลต่อกัน ผ่าน MLflow registry



05 · สถาปัตยกรรม

ทุกส่วนส่งโมเดลต่อกัน ผ่าน MLflow registry



05 . เครื่องมือที่ใช้

ครบทั้ง 9 หน้าตามที่โจทย์กำหนด พร้อมเหตุผลที่เลือก

Version Control → Git + GitHub: ทำงานผ่าน branch + Pull Request ให้ CI ตรวจสอบ merge ทุกครั้ง

Containerization → Docker + docker compose: รันทั้งระบบ 10 บริการได้จากเครื่องเปล่าด้วยคำสั่งเดียว

Data Validation → schema.json + src/validation.py (เขียนเอง): ใส่กฎโดเมนได้ (Credit_Score 300–850)

Experiment Tracking → MLflow: บันทึกครบ 6 อย่าง และเทียบผลข้าม run ได้ในหน้าเว็บ

Model Registry → MLflow alias (@champion / @previous / @challenger): rollback แล้วย้ายป้าย

Pipeline Orchestration → Airflow 3: DAG รันซ้ำได้ทั้งเส้น และ monitoring DAG สั่ง training DAG ได้เอง

Model Serving → Flask ใน Docker: เบา มี /predict, /predict/batch, /health, /metrics

Monitoring → KS / Chi² (เขียนเอง) + Prometheus + Grafana: ดูทั้งคุณภาพการทำนายและสถานะระบบ

CI/CD → GitHub Actions: ตรวจโค้ด (ruff + pytest), ข้อมูล และเกณฑ์โมเดล ทุก PR

05 · เครื่องมือที่ใช้ Library version

ไลบรารี	เวอร์ชัน
Python	>=3.12,<3.13
scikit-learn	>=1.6
pandas	>=2.2
MLflow	>=3.0
Apache Airflow	apache/airflow:3.3.1
Flask	>=3.1
Evidently	==0.7.23

05 · เครื่องมือที่ใช้ใน Container

กลุ่ม Airflow (6 ตัว)

1. **postgres**: ฐานข้อมูลของ Airflow เก็บประวัติ DAG, run และ task
2. **airflow-init**: รันครั้งเดียวตอนยกระบบ สร้างตารางฐานข้อมูลและเตรียมไฟล์ข้อมูล แล้วจบ
3. **apiserver**: หน้าเว็บของ Airflow และ API ที่ task ใช้รายงานผล (พอร์ต 8080)
4. **scheduler**: ตัดสินว่า task ไหนถึงเวลารัน แล้วส่งรัน
5. **dag-processor**: อ่านไฟล์ในโฟลเดอร์ dags/ เป็นระยะ เพื่อให้ Airflow เห็น DAG
6. **triggerer**: ดูแล task ที่รอเหตุการณ์ภายนอก

กลุ่ม model (2 ตัว)

1. **mlflow**: เก็บผลการทดลองทุกรอบ และ model registry ที่มี @champion (พอร์ต 5050)
2. **api**: **Flask API** ให้บริการทำนาย มี /predict, /health, /metrics, /reload (พอร์ต 8000)

กลุ่ม monitoring (2 ตัว)

1. **prometheus**: ดึงตัวเลขจาก /metrics ของ API ทุก 5 วินาที และตรวจกฎแจ้งเตือน 7 ข้อ (พอร์ต 9090)
2. **grafana**: dashboard แสดงสถานะระบบและคุณภาพ model (พอร์ต 3000)

05 · Model ที่เทรนใน pipeline จริง

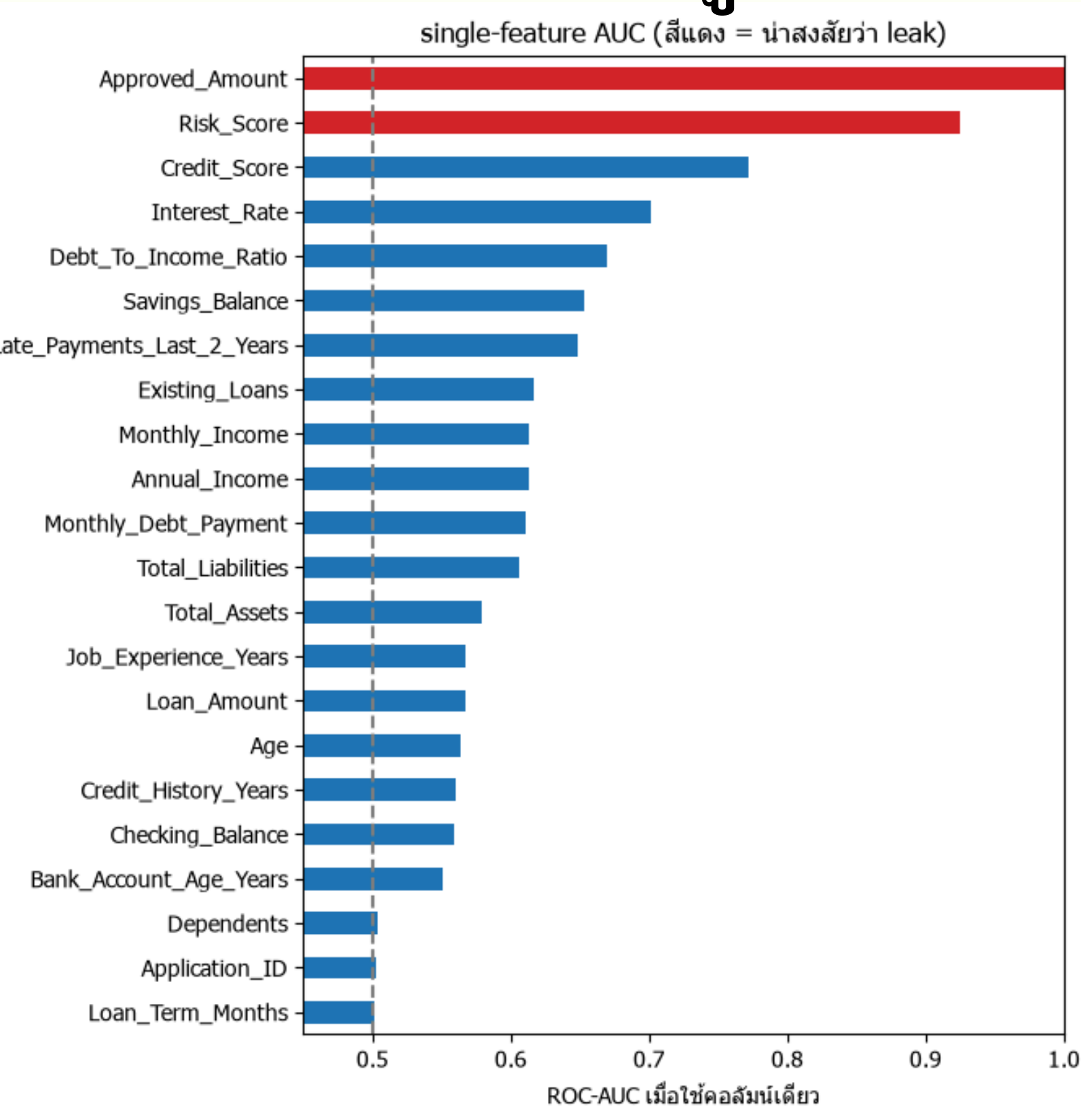
ชื่อใน Pipeline	Model	Hyperparameter ที่ตั้ง
logreg	LogisticRegression	max_iter = 2000 (C ใช้ค่า default คือ 1.0)
logreg_c0.1	LogisticRegression	C = 0.1 , max_iter = 2000
hist_gboost	HistGradientBoostingClassifier	max_iter = 300 , learning_rate = 0.05 , random_state = 42

06 · ข้อมูล

ชุดข้อมูล: [Loan Approval Dataset 2026 \(Kaggle\)](#) มี 30,000 คำขอ หรือ samples , 36 คอลัมน์, license MIT

- คำขอสินเชื่อ 30,000 แถว · 36 คอลัมน์ · เป็นข้อมูลสังเคราะห์จาก Kaggle (license MIT)
- มีข้อมูลผู้สมัคร รายได้ งาน ประวัติเครดิต พฤติกรรมการเงิน รายละเอียดเงินกู้ หลักประกัน และผลอนุมัติ
- เป้าหมาย Loan_Aproved: อนุมัติ 57.6% / ปฏิเสธ 42.4% (สมดุลงพอ ใช้ ROC-AUC ได้ตรง ๆ)
- ไม่มีค่าว่าง ไม่มีแถวซ้ำ แต่ pipeline ยังเติมค่าว่างได้ เพื่อข้อมูลจริง
- ใช้เป็นฟีเจอร์ 27 คอลัมน์ = 36 – เป้าหมาย 1 – leakage 4 – อ่อนไหว/ซ้ำ 3 – ID 1
- และสร้างอัตราส่วนเพิ่ม 2 ตัว (Loan_To_Income, Liability_To_Asset) รวมเข้าโมเดล 29 ตัว

06 • ตรวจสอบข้อมูล



ค่าว่างทั้งหมด: 0
Application_ID ขี้: 0
แถวขี้ (ไม่นับ ID): 0

Loan_Aproved

Approved	17291
Rejected	12709

Loan_Aproved

Approved	0.576
Rejected	0.424

- สิ่งที่เห็น
- ข้อมูลสะอาดมาก (ไม่มีค่าว่าง, ไม่มีแถวขี้) แต่ระบบจริงต้องรับมือค่าว่างได้ เราจึงยังใส่ `SimpleImputer` ไว้ใน pipeline และจะทดสอบกับคำขอที่มีค่าว่างในหัวข้อ 8
 - คลาสค่อนข้างสมดุล (อนุมัติ ~58% / ปฏิเสธ ~42%) จึงใช้ ROC-AUC ได้ตรง ๆ ไม่ต้องทำ oversampling

ตรวจ DATA LEAKAGE: คอลัมน์ไหน "ห้ามใช้"

LEAKAGE คือการเอาข้อมูลที่ **ตอนใช้งานจริงยังไม่มี** มาเทรน โมเดลจะเก่งมากตอนทดสอบ แต่พอใช้จริงแล้วพัง

วิธีหาแบบง่าย: ลองใช้ **คอลัมน์เดียว** ทำนายผลอนุมัติ แล้วดู ROC-AUC ถ้าคอลัมน์เดียวทำได้เกือบ 1.0 แปลว่าคอลัมน์นั้นน่าจะเป็นคำตอบอยู่แล้ว

06 · การจัดการข้อมูล

ตัด 4 คอลัมน์ ที่รู้ค่าหลังธนาคารตัดสินใจ

Column	หลักฐาน Leakage
Approved_Amount	ผู้ถูกปฏิเสธ = 0 ทุกแถว (AUC 1.00)
Default_Risk	กลุ่ม High ถูกปฏิเสธ 95%
Interest_Rate	ธนาคารกำหนด หลังพิจารณา
Risk_Score	AUC ตัวเดียว 0.925 ไม่รู้สูตร

การแบ่งข้อมูล

ข้อมูลทั้งหมด 30,000 แถว
stratified · seed 42 · md5 ของไฟล์ = เวอร์ชันข้อมูล
80% ข้อมูลดี
train 70 / val 15 / test 15

20% pool ทดสอบ
จำลอง production 4 สัปดาห์ + ข้อมูลเสีย
week1_normal
week2_data_drift
week3_concept_drift
week4_concept_drift
bad_data

TRAIN 70% / VALIDATION 15% / TEST 15%
16,800 / 3,600 / 3,600 แถว

20% เป็น POOL (6,000 แถว)

07 • Data validation

ข้อมูลเสีย ทำให้ PIPELINE หยุด ก่อนถึงการเทรน

bad_data.csv ที่ป้อนเข้าไป

- Credit_Score = 920 (เกิน 300–850)
- รายได้ติดลบ
- หมวดใหม่ Freelancer
- ค่าว่าง 15% (เกณฑ์ 5%)
- คอลัมน์ Collateral หาย
- Monthly_Income $\neq$ Annual / 12

Schema สามารถจับข้อมูลที่จำลองการผิดได้ **6/6** ข้อผิดพลาด

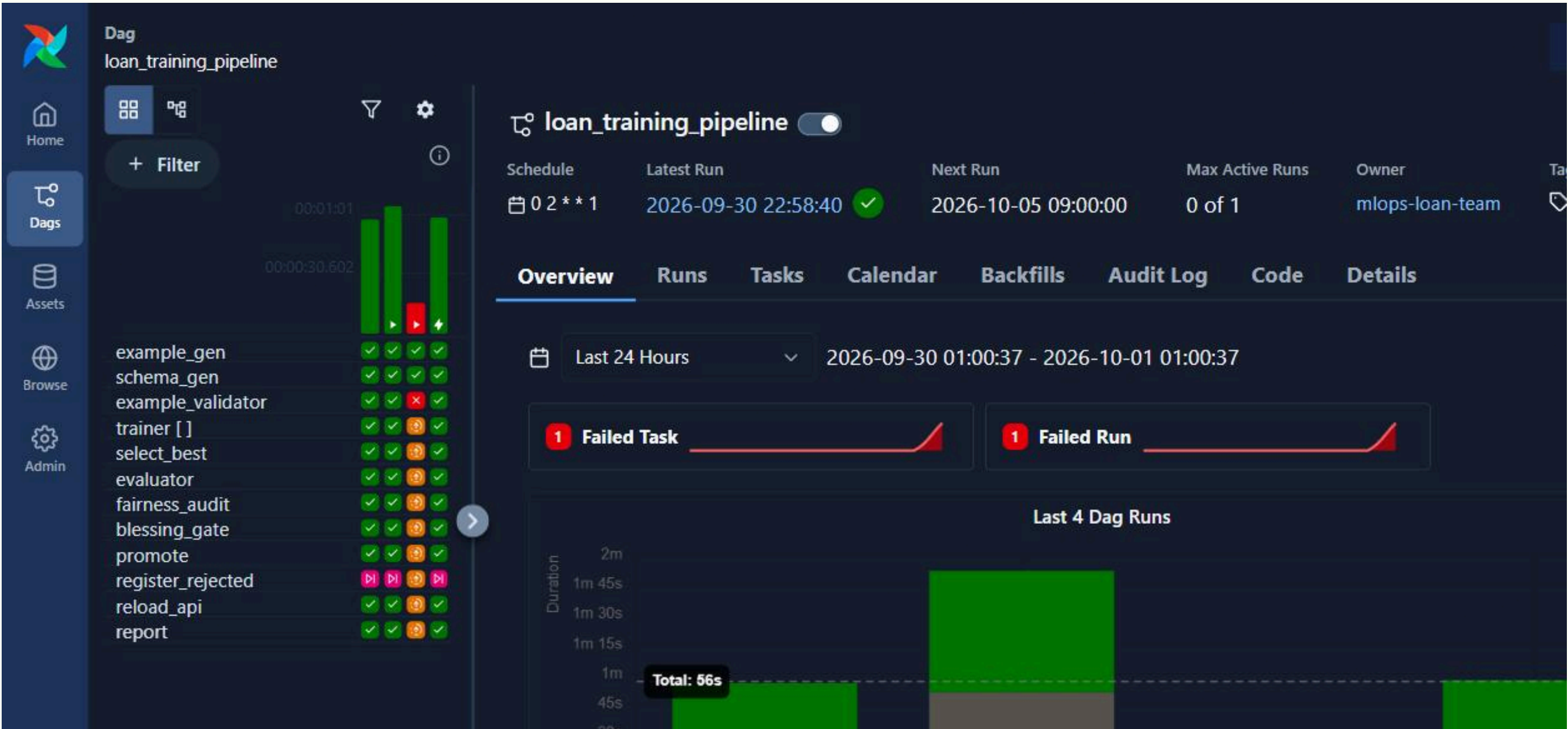
มีการป้องกันข้อมูลเสียอยู่ 2 ที่ โดยใช้ schema ชุดเดียวกัน

- (pipeline train)
 - task train และ deploy ไม่ถูกรัน
 - ALERT ลง alerts.jsonl
- (API)
 - คำขอผิด schema ส่ง HTTP 400 พร้อมรายการ details ที่ระบุชื่อช่องและสาเหตุ ถ้าผิดหลายจุดก็บอกครบทุกจุดในครั้งเดียว

เมื่อข้อมูลผิดกฎใน schema ขั้นตรวจข้อมูล (example_validator) จะหยุด pipeline ทันที โดยไม่ลองซ้ำ

07 • Data validation

รันจริงใน Airflow: example_validator แดง → เทรน / deploy ไม่ถูกรัน + บันทึกลง ALERT

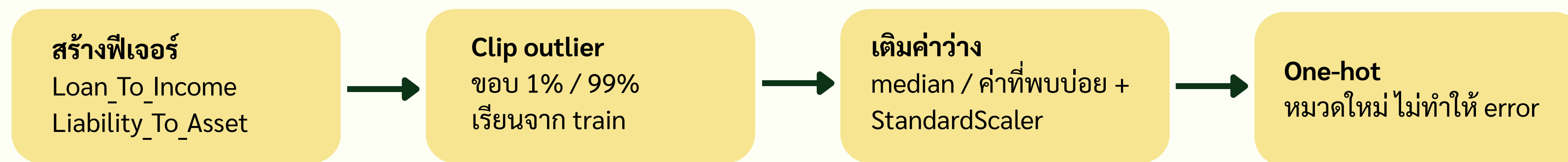


08 • Training-Serving Skew

ปัญหา: ถ้าแปลงข้อมูลตอนให้บริการไม่เหมือนตอนเทรน MODEL จะทายผิดโดยไม่มี ERROR
วิธีแก้: ใช้ PIPELINE ก้อนเดียวกันทั้งสองที่

โค้ดแปลงข้อมูลชุดเดียวกันถูกใช้ทั้ง ตอนเทรน และ ตอนให้บริการ

กลไกป้องกัน 4 ชั้น ไม่ให้เกิด Training-Serving Skew



- เรียนจาก train ครั้งเดียว: ขอบ clip, median, mean/std, รายชื่อหมวด ถูกเก็บไว้ในก้อน ตอนให้บริการแค่นำมาใช้ ไม่คำนวณใหม่
- สภาพแวดล้อมเดียวกัน: library ใน container ล็อกเวอร์ชันตรงกับตอนเทรน และ schema ที่ใช้ตรวจคำขอมจากการเทรนรอบเดียวกับ model
- โค้ดที่เดียว: การแปลงทั้ง 4 ชั้น + model เป็น sklearn Pipeline ก้อนเดียวใน src/features.py
- ไฟล์เดียวกัน: บันทึกทั้งก้อนลง MLflow → API โหลดก้อนนั้นผ่าน @champion และส่งข้อมูลดิบเข้าไปตรง ๆ API ไม่มีโค้ดแปลงข้อมูลของตัวเอง
- พิสูจน์แล้ว: ผลทำนายจากโมเดลที่โหลดจาก Registry ต่างจากตอนเทรนสูงสุด 0.00 → ไม่มี skew

09 • การทดลอง

ค่าที่ใช้ร่วมกันทุก model (ขั้นแปลงข้อมูล)

ขั้น	ค่าที่ตั้ง
ตัดค่าสุดโต่ง(LoanFeatureEngineer)	lower_q = 0.1 , upper_q = 0.99
เติมค่าว่างของตัวเลข(SimpleImputer)	strategy = “median”
เติมค่าว่างของหมวดหมู่(SimpleImputer)	strategy = “most_frequent”
ปรับสเกล(StandardScaler)	ค่า default
OneHotEncoder	handle_unknow = “ignore”
ColumnTransformer	sparse_threshold = 0

Model ที่ทดลอง

การทดลอง	Model	Hyperparameter ที่ตั้ง	จุดประสงค์
E0	DummyClassifier	strategy = “most_frequent”	เส้นฐานขั้นต่ำ
E1	LogisticRegression	max_iter = 2000 +ใส่ Risk_Score	ดูผลของคอลัมน์ที่น่าสงสัยว่าเร็ว
E2	LogisticRegression	max_iter = 2000	baseline จริง
E3	LogisticRegression	c = 0.01 , max_iter = 2000	ลองเพิ่ม regularization
E4	RandomForestClassifier	n_estimators = 300 , min_sample_leaf,n_jobs = -1 , random_state = 42	เทียบกับ model แบบต้นไม้
E5	HistGradientBoostingClassifier	max_iter = 300 , learning_rate = 0.5 , random_state = 42	เทียบกับ boosting

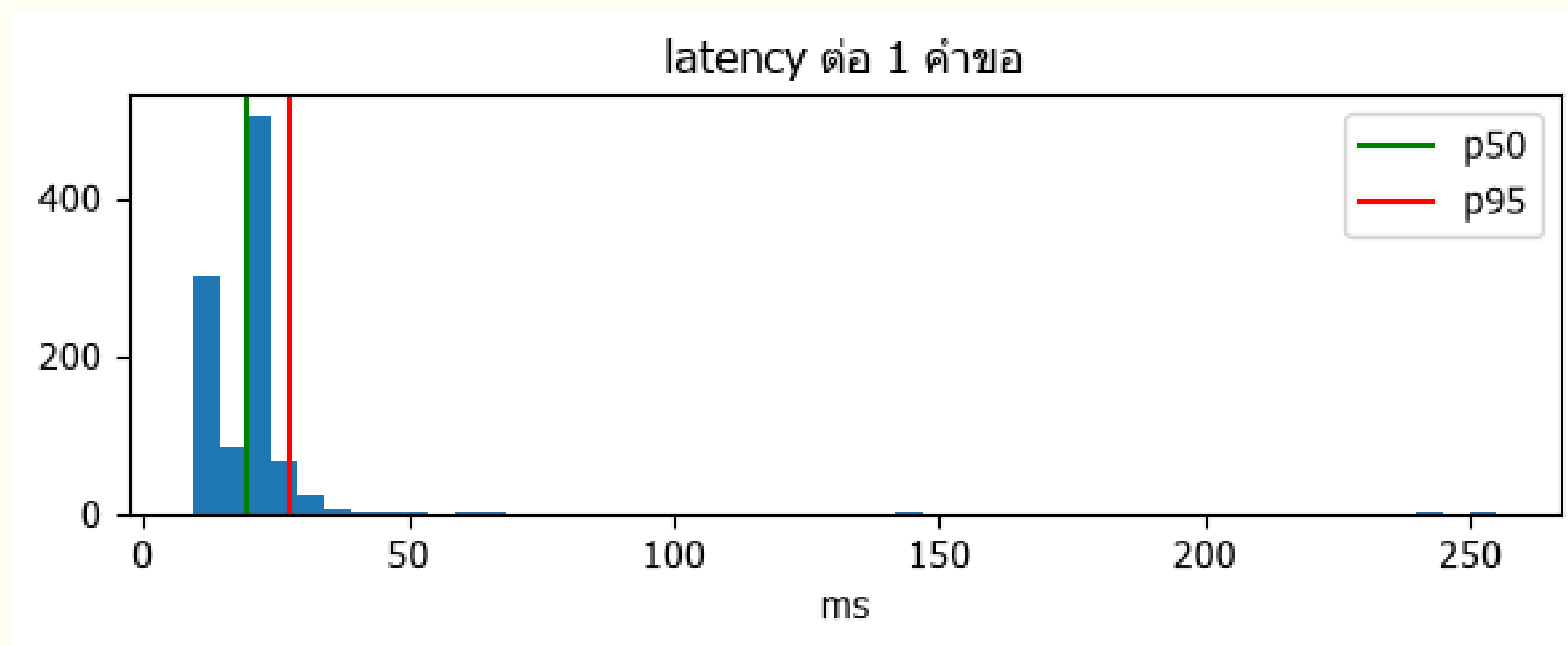
09 • ผลการทดลอง

LOGISTIC REGRESSION แม่นใกล้เคียง โมเดลที่ซับซ้อนกว่า

การทดลอง	Val AUC	เวลาเทรน	ตัดสินใจ
E0 Dummy	0.500	0.09 s	พื้นขั้นต่ำ
E1 LogReg + Risk_Score	0.927	0.11 s	ตัดทิ้ง (เสี่ยง leakage)
E2 LogReg	0.894	0.11 s	เลือกตัวนี้
E3 LogReg C=0.1	0.894	0.12 s	ไม่ต่างจาก E2
E4 Random Forest	0.890	0.52 s	ไฟล์ 50.6 MB ไม่ผ่านด่าน
E5 HistGradientBoosting	0.896	0.45 s	ดีกว่า 0.002 แต่ช้ากว่า 4 เท่า

E0 Dummy = พื้นขั้นต่ำ · Baseline = E2 LogReg (ไม่ใช่ Risk_Score) · ในระบบจริง Airflow เทรน 3 ตัวขนานกัน (logreg, logreg C=0.1, hist_gboost)
select_best: ตัดตัวที่ p95 > 25 ms ก่อน แล้วเลือก logreg เว้นแต่ตัวอื่นชนะ AUC เกิน 0.003 (RF ไม่เข้ารอบ เพราะใหญ่และช้า)
ทำไมตัด Risk_Score ทั้งที่ช่วย +0.033: เป็นคะแนนจากระบบอื่น ที่ไม่รู้สูตร ถ้าสูตรเปลี่ยน โมเดลเราพังตามโดยไม่รู้ตัว

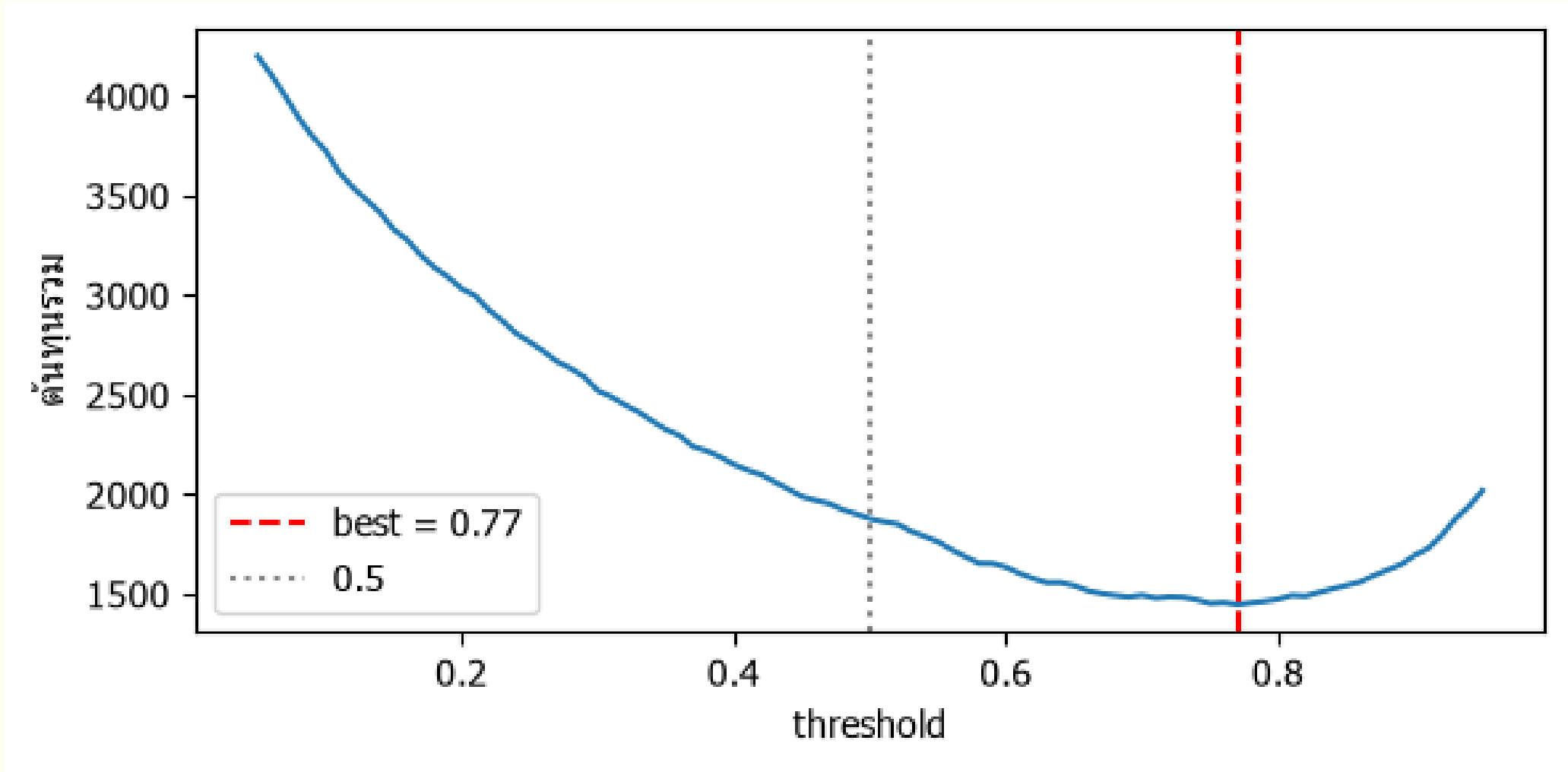
09 • ผลการทดลอง



ทีละคำขอ : P50 = 19.74 MS P95 = 27.74 MS → THROUGHPUT $\approx$ 53 คำขอ/วินาที (1 โพรเซส)
แบบ BATCH: 4,499 คำขอใน 22 MS → $\approx$ 205,530 คำขอ/วินาที

วัดใน notebook (แค่ model อย่างเดียวโดยยังไม่ผ่าน API)

09 · เลือก threshold ตามต้นทุน



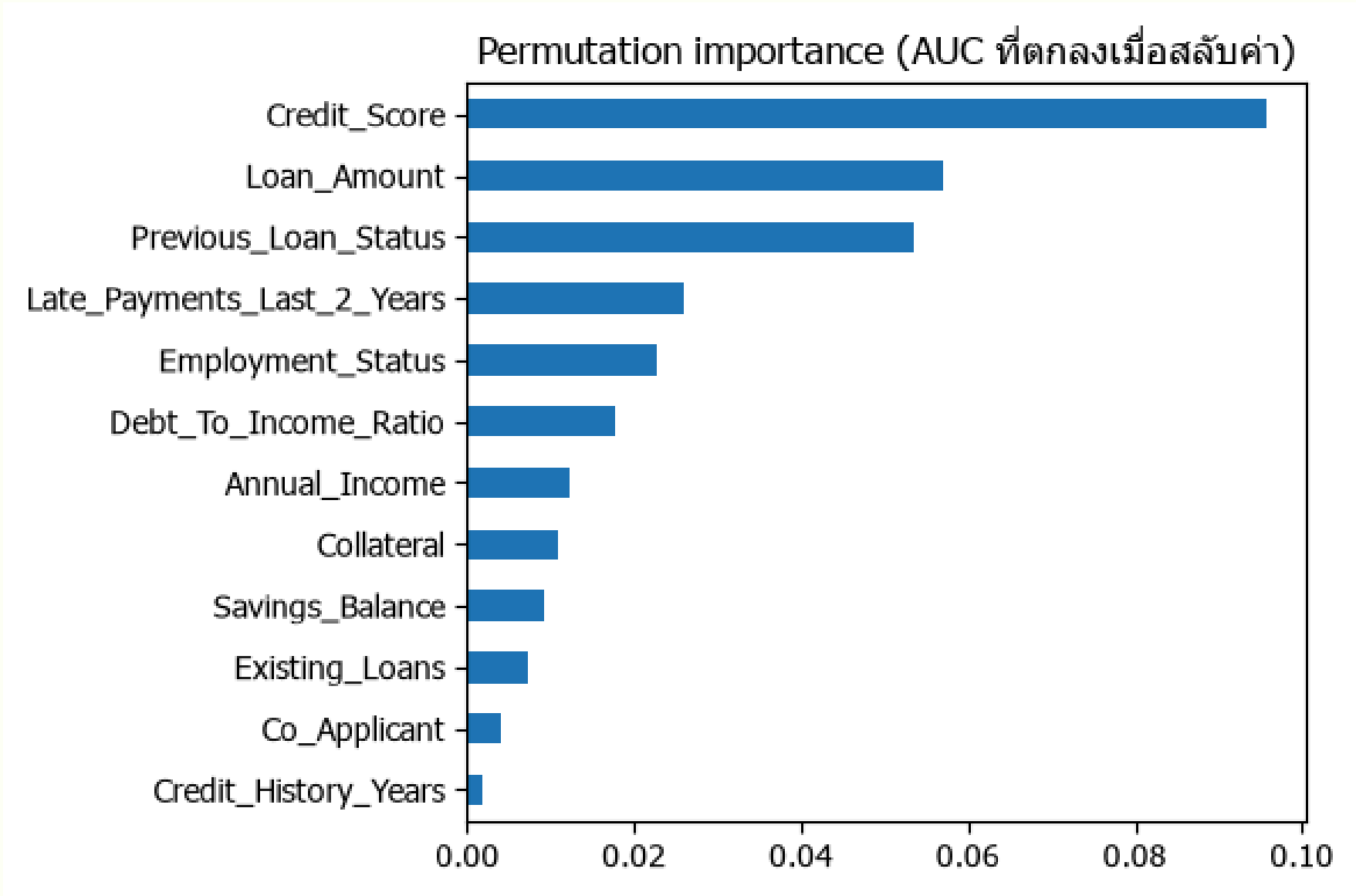
ตัดสินใจอัตโนมัติได้ 78.5% ของคำขอ | ความแม่นยำในส่วนอัตโนมัติ 80.3%
ส่งให้เจ้าหน้าที่ 21.5% ของคำขอ

$$\text{ต้นทุน} = 3 \times \text{อนุมัติผิด} + 1 \times \text{ปฏิเสธผิด}$$

threshold ที่ต้นทุนต่ำสุด = 0.77 ต้นทุนที่ 0.5 = 1879 ที่ 0.77 = 1446
ผลที่ threshold ใหม่: {'roc_auc': 0.894, 'f1': 0.7586, 'accuracy': 0.76, 'recall_rejected': 0.904}

0.77 เป็นค่าจาก notebook ส่วนระบบจริงเลือกใหม่ทุกครั้งที่เทรน

09 . อธิบายผล: โมเดลดูอะไร



ดันไปทาง '**ปฏิเสธ**' มากสุด:

cat_Previous_Loan_Status_Defaulted	-2.616
cat_Employment_Status_Unemployed	-1.783
num_Late_Payments_Last_2_Years	-0.669
num_Loan_To_Income	-0.600
num_Debt_To_Income_Ratio	-0.472

ดันไปทาง '**อนุมัติ**' มากสุด:

cat_Employment_Status_Business Owner	0.511
cat_Previous_Loan_Status_No Previous Loan	0.567
cat_Previous_Loan_Status_Active	0.570
num_Credit_Score	1.137
cat_Previous_Loan_Status_Paid Off	1.357

10 • Tracking + Registry

ย้อนเวอร์ชันได้ ด้วยการย้ายป้าย alias

@champion
ตัวที่ API ให้บริการอยู่

@previous
champion ก่อนหน้า ไว้ rollback

@challenger
ตัวที่ไม่ผ่านด่าน เก็บเป็นหลักฐาน

- 1.blessing_gate ตรวจสอบ gating ครบ 6 ข้อ
- 2.ผ่าน → promote และ @previous = ตัวเดิม
- 3.ไม่ผ่าน → register_rejected + ALERT
- 4.rollback.py → @champion กลับ → API /reload

ทุก run บันทึกลง mlflow ครบ 6 อย่าง

git commit

metrics

data md5

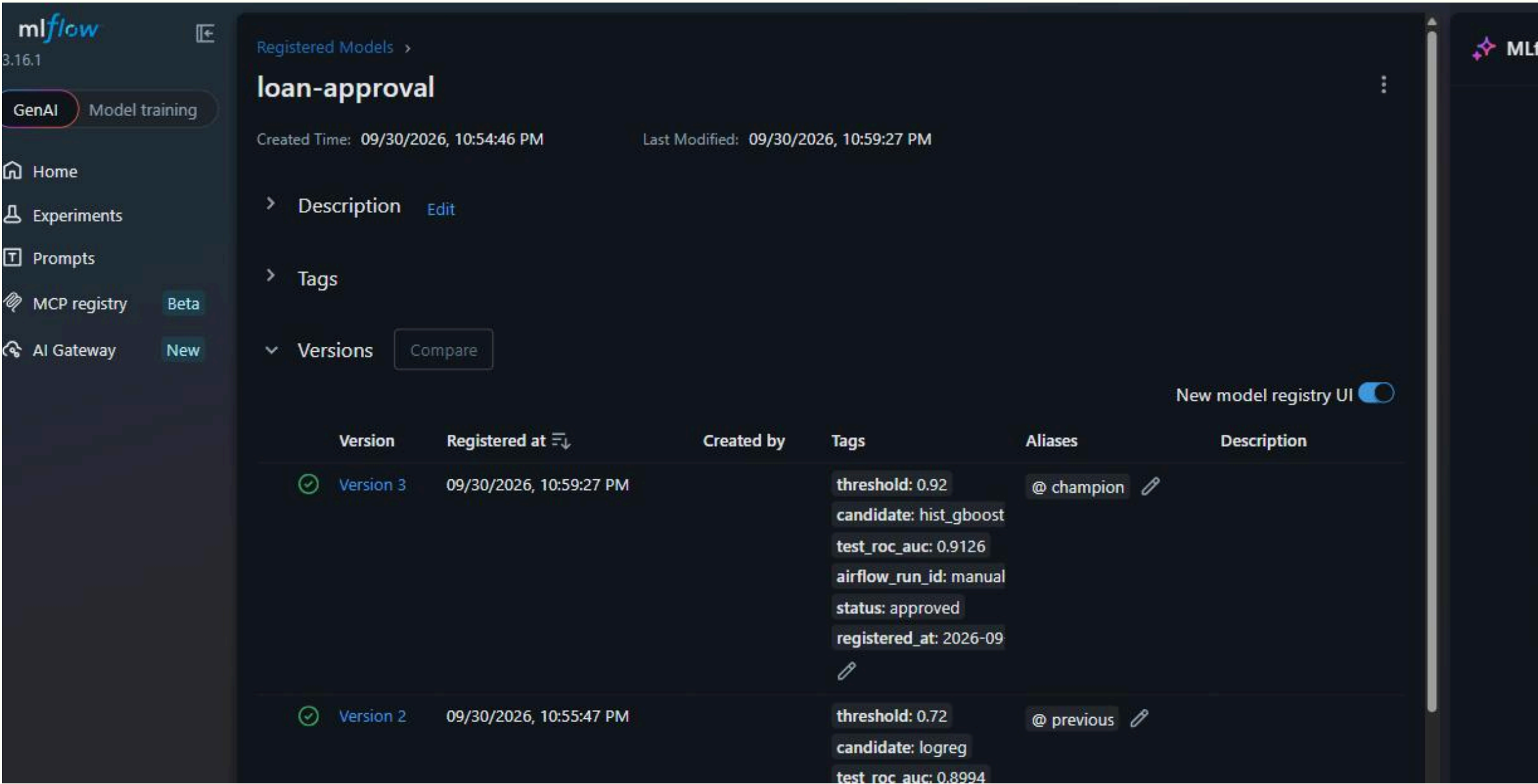
model + schema

hyperparameters

environment + uv.lock

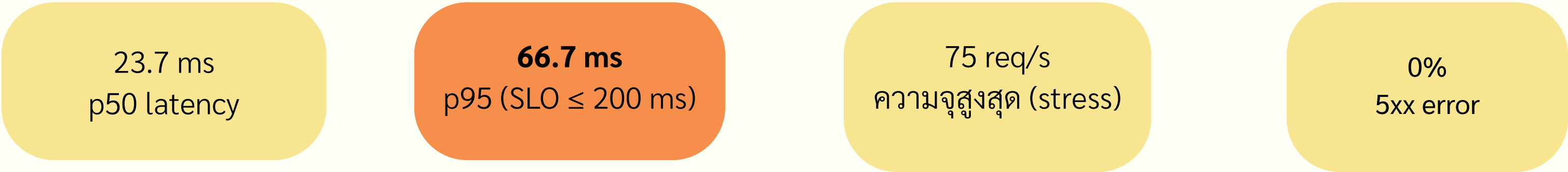
10 • Model Registry (MLflow)

หลักฐานจริง: v3 hist_gboost = @champion · v2 logreg = @previous (rollback ด้วย scripts/rollback.py)

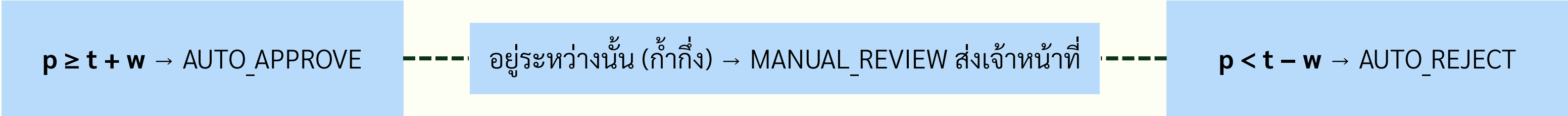


11 • Serving + SLO

Flask API ใน Docker (/predict · /predict/batch · /health · /metrics) · SLO: $p_{95} \leq 200\text{ ms}$ ที่ 20 req/s และ $5xx \leq 1\%$
ผลวัดจริง: ผ่านทั้งสองข้อ
"วัดผ่าน API จริง"



รูปแบบ: real-time เพราะลูกค้ารอผลทันที + cascade ส่งเคสที่ไม่มั่นใจให้เจ้าหน้าที่ ·
 $w = 0.10$ แต่ไม่เกินครึ่งของระยะที่เหลือถึง 0/1 (เช่น $t = 0.72 \rightarrow w$ ฝั่งอนุมัติ = 0.04)



บทเรียน: HistGradientBoosting ใช้ ~35 ms ต่อคำขอ ทำ p95 พุ่งถึง 195 ms → ผูกด้าน latency $\leq 25\text{ ms}$ กับ SLO และเลือกโมเดลโดยดูความเร็วด้วย

12 • Monitoring + Retrain

แยก data drift กับ concept drift ได้ แล้ว retrain เอง

Batch	Drift share	AUC	ผล (AUC ตอน deploy 0.899)
W1 ปกติ	0%	0.895	ไม่มี alert
W2 เศรษฐกิจถดถอย	63%	0.918	ALERT data drift • ไม่ retrain
W3 นโยบายใหม่	0%	0.801	ALERT concept drift → retrain อัตโนมัติ
W4 หลัง retrain	0%	0.931	ไม่มี alert (โมเดล v3)

Data drift = $P(X)$ เปลี่ยน
ตรวจด้วย : KS test (ตัวเลข), Chi-square (หมวดหมู่) ไม่ต้องรอเฉลี่ย
เกณฑ์แจ้งเตือน: $p < 0.01$ ในเกิน 20% ของฟีเจอร์
จำลองด้วย: เศรษฐกิจถดถอย: สุ่มแบบถ่วงน้ำหนักให้คน คะแนน/รายได้
 ต่ำ มากขึ้น ($P(y|X)$ เท่าเดิม)

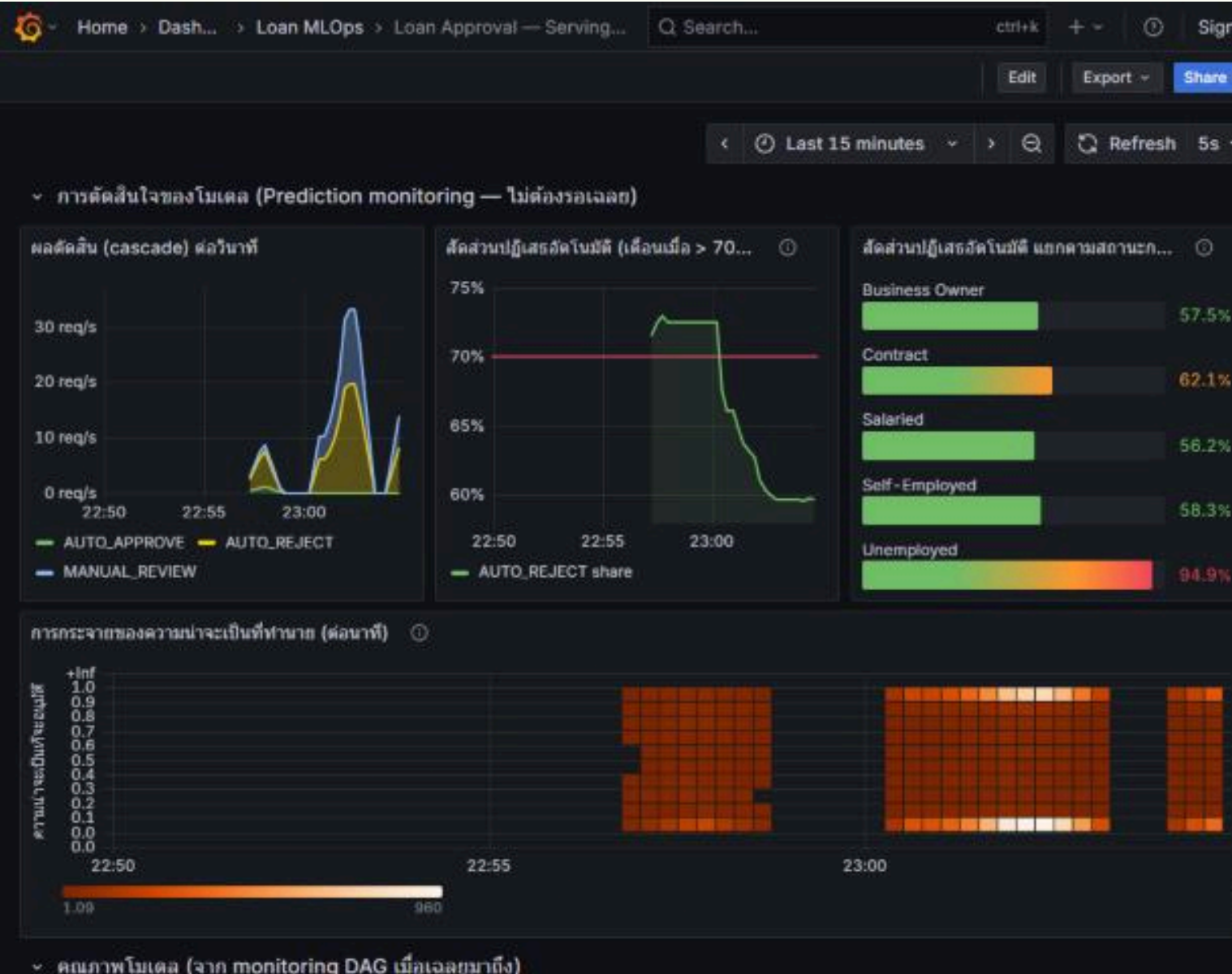
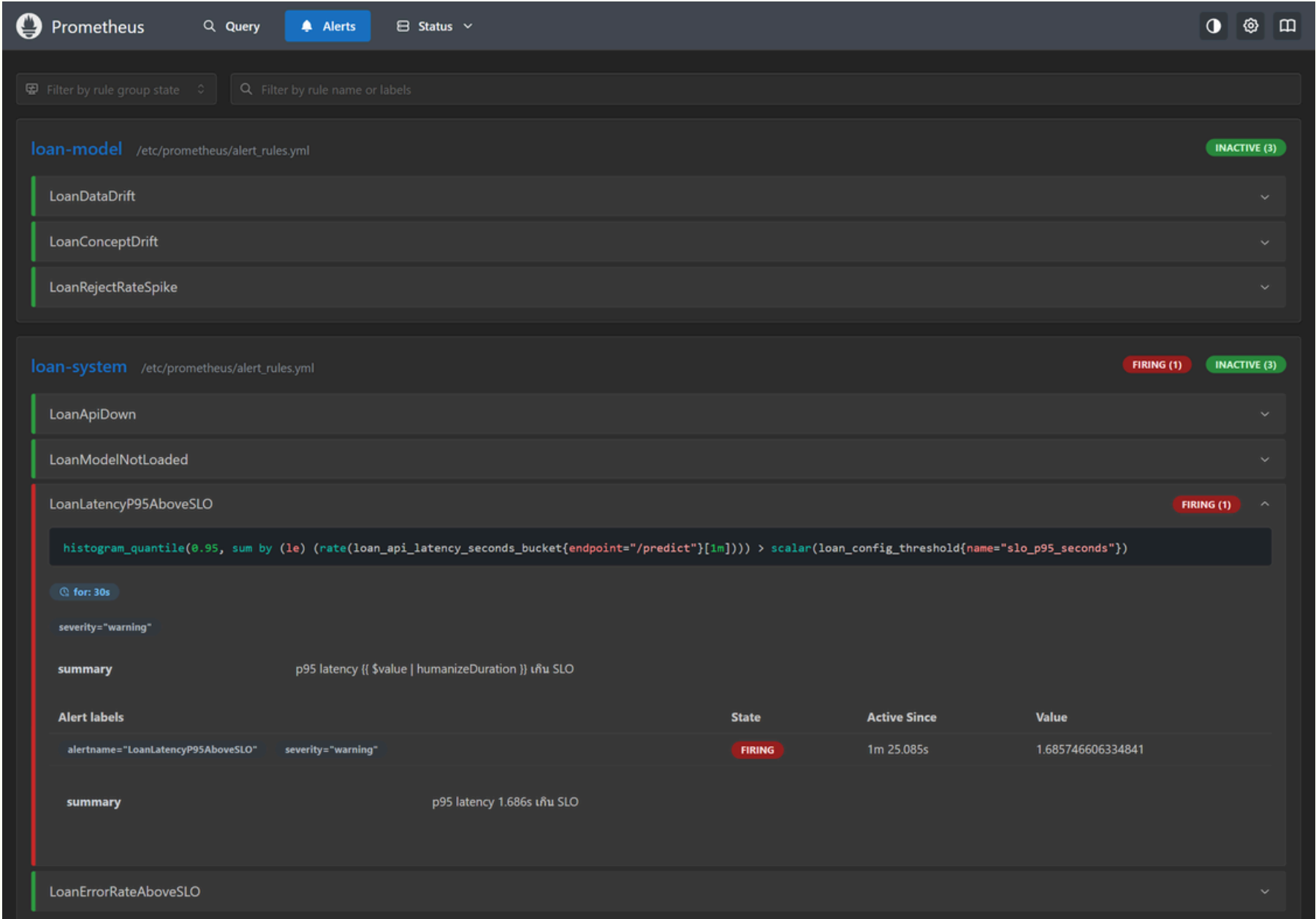
Concept drift = $P(y|X)$ เปลี่ยน
ตรวจด้วย : AUC บนเฉลี่ยจริง เทียบ AUC ตอน deploy
เกณฑ์การแจ้งเตือน : AUC ตกเกิน 0.05
จำลองด้วย : ธนาคารเลิกปล่อยกู้ระยะเกิน 120 เดือน (เปลี่ยน label
 ไม่แตะข้อมูลเข้า)

12 · นโยบาย Retrain

เงื่อนไข	การกระทำ
AUC ตกเกิน 0.05 เมื่อเฉลี่ยมาถึง	Monitoring DAG สั่ง training DAG ด้วย TriggerDagRunOperator เทรนด้วยข้อมูลล่าสุดที่มีเฉลี่ย (sliding window) เพราะเฉลี่ยเก่าไม่ตรงนโยบายแล้ว
Data drift เกิน 20% แต่ AUC ปกติ	แจ้งเตือนทีมให้ตรวจ ยังไม่ retrain
ข้อมูลเสีย	หยุด ห้าม retrain แจ้งเตือน
ตามรอบ	training DAG ทุกวันจันทร์ 02:00 UTC

12 · หลักฐาน Monitoring

ซ้าย: Prometheus กฎ LoanLatencyP95AboveSLO firing ตอน stress test · ขวา: Grafana ผลตัดสินแยกกลุ่ม + สัดส่วนปฏิเสธ

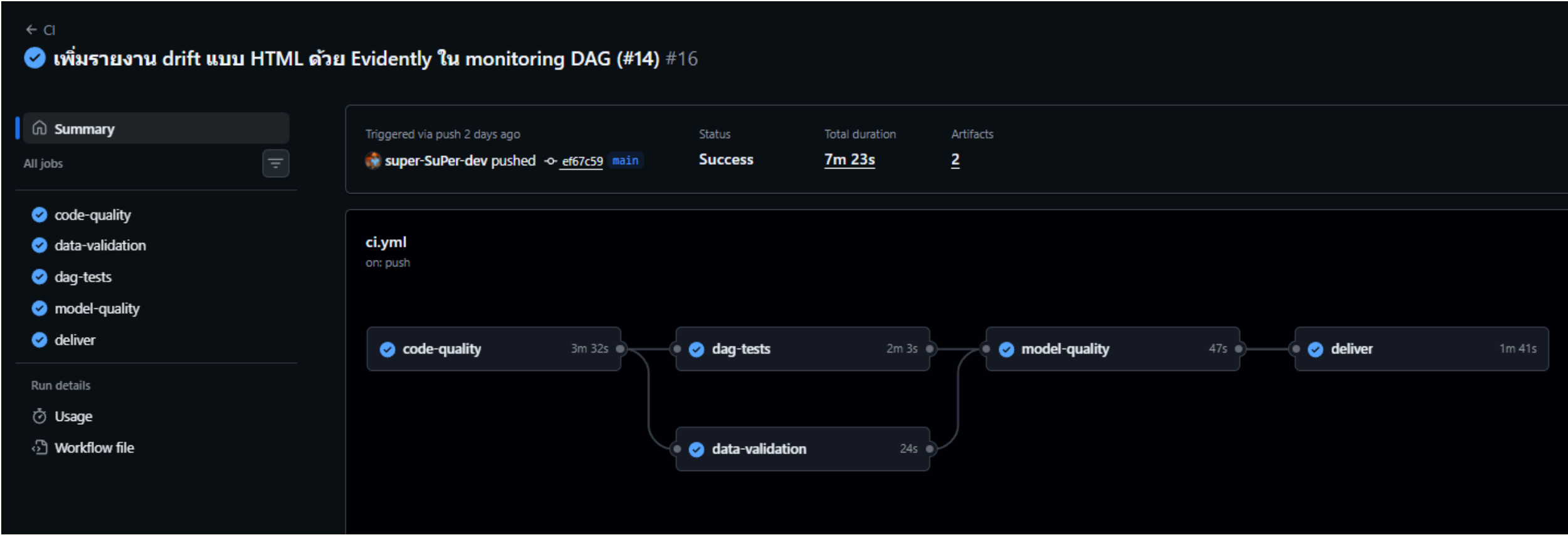


13 · CI/CD

CI ตรวจ 3 ด้าน ทุก Pull Request ก่อน merge

Job	งานด้าน	ตรวจอะไร	รอบแรก
code-quality	คุณภาพโค้ด	ruff + pytest 14 เทสต์	ผ่าน 64 วินาที
data-validation	ข้อมูล	ข้อมูลจริงผ่าน schema · ข้อมูลเสียต้องถูกจับ	ผ่าน 24 วินาที
model-quality	โมเดล	เทรนนข้อมูลจริง ต้องผ่านทุกข้อ + fairness	ผ่าน 33 วินาที
dag-tests	pipeline	ทดสอบ DAG ใน image Airflow	ผ่าน 97 วินาที
deliver	ส่งมอบ	build API + smoke test → push ghcr.io	ผ่าน 98 วินาที

13 • CI/CD หลักฐานครั้งที่ผ่าน

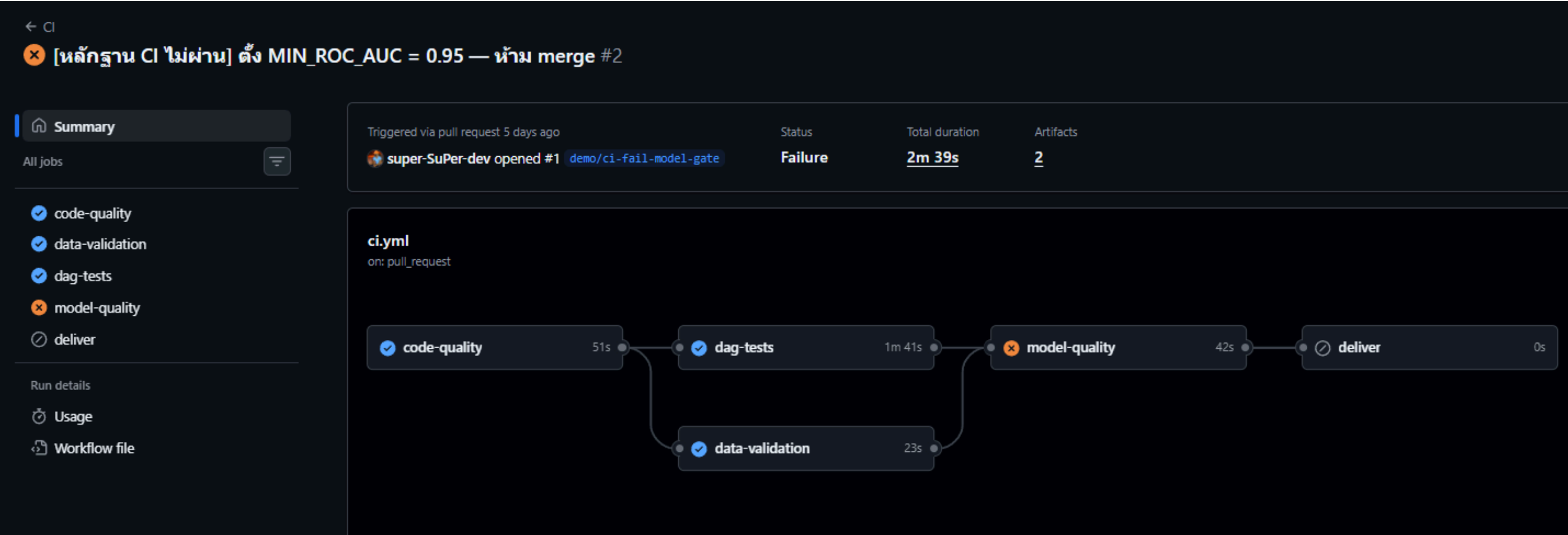


data-validation summary	
✓ Data validation (ข้อมูลจริง)	
รายการ	ผล
จำนวนแถว historical	24000
data md5	58c72146159e6c6a289faa34e4901dc8
train/val/test ผ่าน schema	ผ่าน
week_1_normal.csv ผ่าน schema	ผ่าน
week_2_data_drift.csv ผ่าน schema	ผ่าน
week_3_concept_drift.csv ผ่าน schema	ผ่าน
bad_data.csv ถูกจับได้	6 ปัญหา

model-quality summary	
✓ Model quality gate (ข้อมูลจริง)	
รายการ	ผล
โมเดลที่เลือก	logreg
threshold	0.72
ROC-AUC (≥ 0.85)	0.8994
Recall ปฏิเสธ (≥ 0.8)	0.8885
p95 latency ms (≤ 25.0)	18.20
ขนาดโมเดล MB (≤ 50.0)	0.010
Fairness: ส่วนต่างอัตราอนุมัติ (≤ 0.1)	0.0171
ด้าน roc_auc	ผ่าน
ด้าน recall_rejected	ผ่าน
ด้าน p95_latency	ผ่าน
ด้าน model_size	ผ่าน
ด้าน vs_champion	ผ่าน

13 · CI/CD

หลักฐานครั้งที่ ไม่ผ่าน



model-quality summary	
✖ Model quality gate (ข้อมูลจริง)	
รายการ	ผล
โมเดลที่เลือก	logreg
threshold	0.72
ROC-AUC (≥ 0.95)	0.8994
Recall ปฏิเสธ (≥ 0.8)	0.8885
p95 latency ms (≤ 25.0)	18.26
ขนาดโมเดล MB (≤ 50.0)	0.010
Fairness: ส่วนต่างอัตราอนุมัติ (≤ 0.1)	0.0171
ด้าน roc_auc	ไม่ผ่าน
ด้าน recall_rejected	ผ่าน
ด้าน p95_latency	ผ่าน
ด้าน model_size	ผ่าน
ด้าน vs_champion	ผ่าน

[Job summary generated at run-time](#)

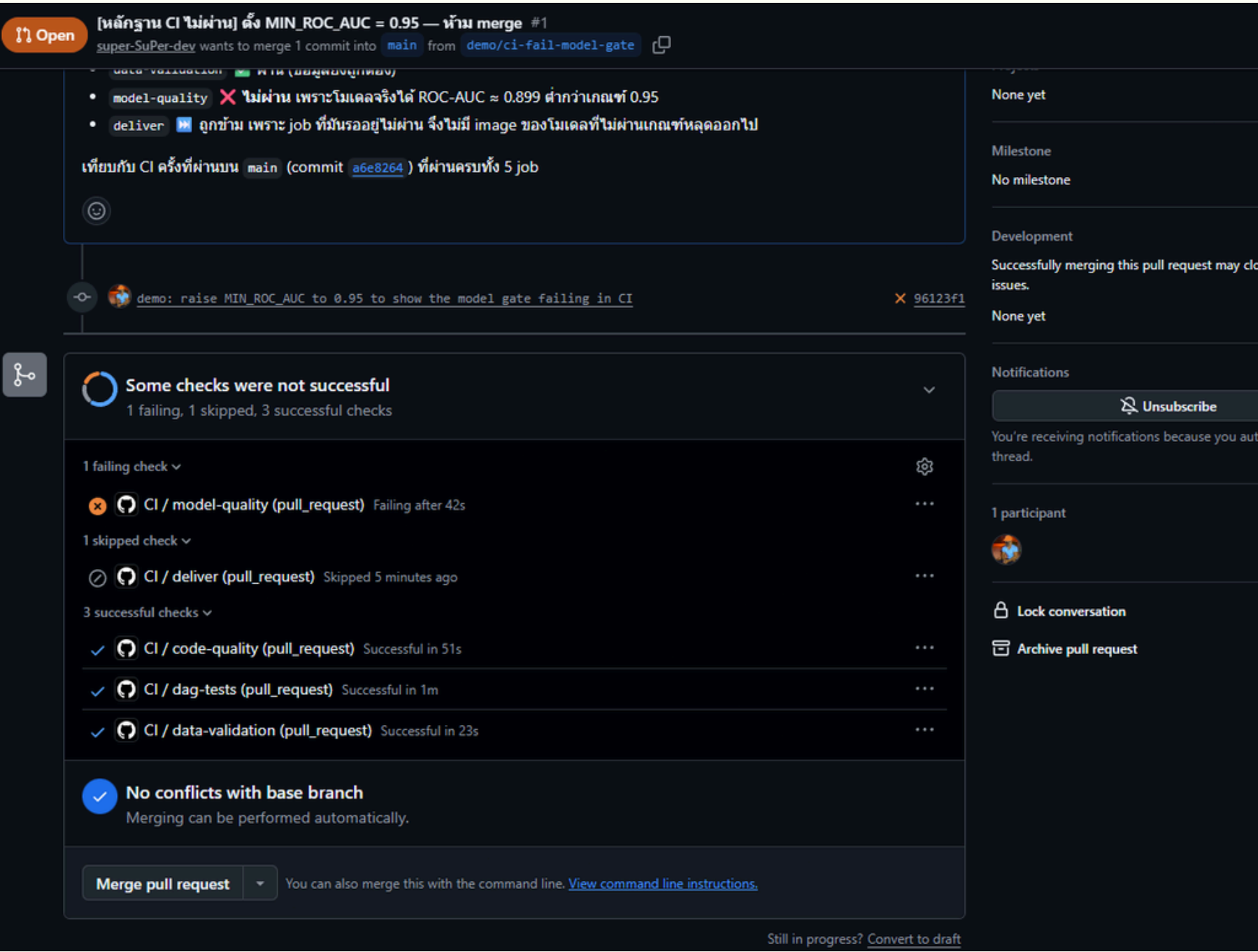
PR ที่ตั้ง

MIN_ROC_AUC = 0.95

model-quality ขึ้นแดง (AUC 0.899 ไม่ถึง) เลย merge ไม่ได้

14 · Git + รันจากเครื่องเปล่า

ทำงานผ่าน branch + Pull Request เช่น PR #1 จาก demo/ci-fail-model-gate → main ถูก CI บล็อก



- ทุกงานทำบน branch แยก แล้วเปิด PR เข้า main
- CI ต้องผ่านก่อน merge (ภาพ: model-quality แดง → merge ไม่ได้)
- รันจากเครื่องเปล่าตาม README:
- git clone → uv sync → docker compose up -d --build
- กด Trigger loan_training_pipeline ใน Airflow ปุ่มเดียว ได้ครบตั้งแต่ข้อมูลดิบจนถึง API
- ล็อกเวอร์ชันไลบรารีด้วย uv.lock + seed 42 → รันซ้ำได้ผลเดิม

15 • สาริต

ทั้งระบบ ด้วยคำสั่งเดียว

```
uv run python scripts/demo.py
```

1. เทรนรอบแรก → champion
2. ข้อมูลเสีย → pipeline หยุด + ALERT
3. W1 ปกติ → ไม่มี alert
4. W2 data drift → แจ้งเตือน ไม่ retrain
5. W3 concept drift → retrain เอง
6. W4 → AUC กลับมา 0.93
7. Load test → เทียบ SLO

ข้อจำกัด

- ข้อมูลสังเคราะห์ ไม่มีเวลา → drift เป็นการจำลอง
- โมเดลหลังนโยบายใหม่ เทรนจาก 1,500 แถว
- Flask โปรเซสเดียว ≈ 75 req/s

<https://github.com/super-SuPer-dev/loan-mlops>



THANK YOU